KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

# VENDOR SUPPLY TRACKING AND ORDER PROCESSING SYSTEM

Submitted by

**Pranav Krish**

**JD 25BIT135**

Under the Guidance of

**MUGILAN R**

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

**APRIL 2026**

Semester: **II**

# DECLARATION

I hereby declare that the project entitled "**Parking Slot Allocation and Vehicle Entry Tracking System"** submitted in partial fulfillment of the requirements for the course **Java Programming** is a **record of my original work** carried out under the guidance of **MUGILAN R**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

(Pranav Krish JD)

# CERTIFICATE

This is to certify that the project entitled **"Parking Slot Allocation and Vehicle Entry Tracking System"** is a **bona fide work** carried out by **Pranav Krish JD B.Sc.IT 'B' (2025-2028)**, **INFORMATION TECHNOLOGY**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

**Place:COIMBATORE**

**Date:**

Submitted for the Viva-Voce Examination held on _____

**Guide:**
(Technical Trainer, KGCAS)

_____

**Head of the Department**
(Technical Head, KGCAS)

_____

## TABLE OF CONTENTS


| S. No. | Topics | Page. No. |
|---|---|---|
| **1** | **Introduction** | |
| 1.1 | Objective | |
| 1.2 | Overview | |
| 1.3 | Features | |
| **2** | **Program Analysis** | |
| 2.1 | Problem Definition | |
| 2.2 | Project Overview | |
| 2.3 | Scopes and Limitations | |
| **3** | **System Design** | |
| 3.1 | Modules | |
| **4** | **Methodology** | |
| 4.1 | Problem Analysis | |
| 4.2 | System Design | |
| 4.3 | Program Development | |
| 4.4 | Testing and Validation | |
| 4.5 | Result and Documentation | |
| 4.6 | Algorithm | |
| 4.7 | Flowchart | |
| **5** | **Source Code** | |
| **6** | **Input and Output Screens** | |
| **7** | **Discussion and Conclusion** | |
| 7.1 | Discussion | |
| 7.2 | Conclusion | |
| **8** | **References** | |

# ABSTRACT

The Parking Slot Allocation and Vehicle Entry Tracking System is a Java-based application designed to efficiently manage parking spaces and monitor vehicle entry and exit in parking facilities. In many small and medium establishments, parking management is often handled manually, leading to issues such as improper slot allocation, congestion, lack of tracking, and security concerns.

This system automates the process of assigning parking slots, recording vehicle entry and exit details, and tracking slot availability in real time. It allows users to register vehicles, allocate available parking slots, update parking status, and monitor vehicle movement in an organized and systematic manner.

The application is developed using Core Java concepts such as classes, objects, methods, loops, and exception handling. It can be implemented as a console-based application or further enhanced with a graphical user interface using Java Swing.

The main objective of this system is to improve parking efficiency, reduce manual effort, minimize errors, and ensure better space utilization and security. This project demonstrates how Java can be applied to develop practical, real-world solutions for parking management systems.

# 1. INTRODUCTION

In today's fast-growing urban environment, efficient parking management systems play a crucial role in reducing congestion and ensuring smooth vehicle movement. Traditionally, many parking facilities rely on manual methods to allocate parking slots and track vehicle entry and exit. These methods include maintaining records on paper or using basic systems, which can be time-consuming and prone to errors.

Manual systems often lead to problems such as incorrect slot allocation, difficulty in tracking parked vehicles, delays in entry and exit management, and lack of proper record maintenance. As the number of vehicles increases, managing parking manually becomes more complex and inefficient.

With the advancement of computer technology, automated parking management systems have become essential for improving efficiency and accuracy. A computerized system helps in allocating parking slots, tracking vehicle movement, and maintaining records in an organized manner. It also ensures quick access to information whenever required.

The Parking Slot Allocation and Vehicle Entry Tracking System is a Java-based application designed to automate the process of managing parking spaces and tracking vehicle entry and exit. The system allows users to register vehicles, assign parking slots, monitor slot availability, and maintain records of vehicle movement. It provides a structured summary of parking details, helping in proper management and security.

This project is developed using Core Java programming concepts such as classes, objects, methods, loops, and conditional statements. The application can be implemented as a console-based program or enhanced using Java Swing for a graphical user interface.

The main objective of this system is to reduce manual effort, improve accuracy, and provide a faster and more efficient way of managing parking operations. This project also demonstrates how Java programming can be applied to develop practical real-world applications.

---

## 1.1 Objective

The main objective of the Parking Slot Allocation and Vehicle Entry Tracking System is to develop a simple and efficient application using Core Java that automates the process of parking management.

The specific objectives of this project are:

- To design and develop a Java-based system for managing parking slots and vehicle tracking
- To automate the process of vehicle entry and exit recording

- To reduce manual errors in slot allocation and record maintenance
- To provide a user-friendly system for managing parking operations
- To maintain proper records of vehicle entry, exit, and slot usage

---

## 1.2 Overview

The Parking Slot Allocation and Vehicle Entry Tracking System is a menu-driven application developed using the Java programming language. The system provides a digital platform where users can manage parking slots and track vehicle movement efficiently.

The system allows the user to register vehicles and allocate available parking slots. It records the entry time and exit time of vehicles and updates slot availability automatically. The user can view the current status of parking slots and track parked vehicles easily.

The system simplifies parking management by eliminating manual work and reducing errors. It also helps in maintaining proper records of vehicle entry and exit in an organized manner.

The application can be developed as a console-based program or enhanced with a graphical user interface using Java Swing.

This project demonstrates the practical use of Java programming in developing real-world applications for environments such as parking lots, malls, office buildings, and public parking facilities.

---

## 1.3 Features

- **Vehicle Registration** – Allows the user to enter and store vehicle details such as vehicle number and type
- **Parking Slot Allocation** – Assigns available parking slots automatically to vehicles
- **Entry and Exit Tracking** – Records vehicle entry and exit time
- **Slot Availability Management** – Displays available and occupied parking slots
- **Automatic Status Update** – Updates slot status when vehicles enter or leave
- **Record Maintenance** – Maintains records of all parked vehicles and their details
- **User-Friendly Interface** – Provides a simple menu-driven interface
- **Fast Processing** – Reduces time required for managing parking operations

---

# 2.PROGRAM ANALYSIS

## Program analysis is an important phase in software development where the problem is studied in detail and system requirements are identified. This stage helps in understanding how the system should function and what features are necessary to solve the problem effectively.

The analysis phase focuses on identifying issues in the existing parking system and proposing a computerized solution that improves efficiency and accuracy.

---

## 2.1 Problem Definition

In many parking areas, vehicle entry tracking and slot allocation are performed manually using registers or basic systems. This traditional method creates several challenges.

Some common problems include:

- Difficulty in tracking parked vehicles
- Improper parking slot allocation
- Time-consuming entry and exit recording
- Lack of real-time slot availability information
- Poor record management
- Increased chances of human errors

Manual systems increase workload and may lead to confusion, delays, and security issues.

Therefore, there is a need for an automated system that can efficiently manage parking slots and track vehicle movement accurately.

The Parking Slot Allocation and Vehicle Entry Tracking System is designed to solve these problems by providing a computerized solution that automates the entire process.

---

## 2.2 Project Overview

The Parking Slot Allocation and Vehicle Entry Tracking System is a Java-based application developed to simplify parking management.

The system provides a menu-driven interface where users can register vehicles, allocate parking slots, and track vehicle entry and exit efficiently. It automatically updates slot availability and maintains records of all activities.

The system includes the following functionalities:

- Registering vehicle details
- Allocating parking slots
- Recording vehicle entry and exit
- Updating slot availability automatically
- Viewing parking status and records

This project is implemented using Core Java programming concepts and can be developed as either a console-based application or a Java Swing GUI application.

---

## 2.3 Scope and Limitations

### Scope

The scope of the Parking Slot Allocation and Vehicle Entry Tracking System includes:

- Managing parking slot allocation
- Tracking vehicle entry and exit
- Monitoring parking slot availability
- Maintaining parking records
- Improving parking efficiency
- Demonstrating Java programming

concepts The system can be used in:

- Shopping malls
- Office buildings
- Residential complexes
- Public parking areas
- Educational institutions

---

### Limitations

- The system may support only a limited number of parking slots depending on implementation
- It may not include advanced features like online booking or payment integration
- Requires manual input of vehicle details
- Security features may be basic in the initial version
- Not integrated with real-time sensors or automated gate systems

# 3.SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The **Vendor Supply Tracking and Order Processing System** is designed as a menu-driven application where users can manage vendor details, track supplies, and process orders efficiently.

The system is divided into several modules to perform specific tasks. Each module is responsible for handling a particular function of the system.

### 3.1 Modules

The main modules of the **Vendor Supply Tracking and Order Processing System** are explained below:

**1. Vendor Management Module**

This module is responsible for storing and managing vendor details.

Example Vendor:

1. Ravi Traders
2. Kumar Suppliers
3. ABC Distributors

The vendor list is maintained so that the user can easily manage and refer to vendor information.

## **2.**Supply Tracking Module

This module handles the recording of items supplied by vendors. Functions of this module include:
• Recording item name
• Storing quantity and price
• Maintaining supply details

This module helps in tracking the goods received from different vendors.

## 3. Order Processing Module

This module allows the user to select items and process orders.

Functions of this module include:

• Selecting items for order
• Entering quantity of items
• Processing multiple items in a single order

The system allows users to handle multiple items before generating the final order summary.

## 4. Billing and Calculation Module

This module performs all calculations related to order processing.

The system calculates:

• Total price of each item
• Overall order

amount Example

Calculation:

Item Total = Price × Quantity
Final Amount = Sum of all item totals

## 5. Order Summary Module

This module generates the final output of the order.

The summary includes:

• Item name
• Quantity
• Price
• Total amount

Example

Output:

------ ORDER SUMMARY ------

Item Qty Price
Rice 2 100
Oil 1 120

Total 220

This module provides a clear and structured view of the processed order.

## 6. Exit Module

This module allows the user to exit the application safely after completing the process. When the user selects the exit option, the system terminates the program.

## 4.METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the Parking Slot Allocation and Vehicle Entry Tracking System, a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

---

## 4.1.Problem Analysis

In this phase, the problem is studied and analyzed in detail. The requirements of the system are identified based on parking management needs.

The main problems identified in manual parking systems are:

- Difficulty in tracking vehicle entry and exit
- Improper allocation of parking slots
- Time-consuming manual record maintenance
- Lack of real-time slot availability information
- Increased chances of human error

Based on these problems, system requirements were defined to develop an automated application that simplifies parking slot allocation and vehicle tracking.

---

## 4.2.System Design

In this stage, the overall structure of the system is designed before coding begins.

The system is divided into modules such as:

- Vehicle Registration Module
- Parking Slot Allocation Module
- Entry Tracking Module
- Exit Tracking Module
- Slot Status Display Module
- Record Management Module
- Exit Module

The program flow and structure are designed in such a way that users can easily interact with the system through a simple menu-driven interface.

---

## 4.3Program Development

After designing the system, the program is implemented using the Java programming language.

The following steps were followed during development:

- Creation of classes and methods
- Implementation of menu-driven logic
- Writing code for slot allocation and tracking vehicle entry/exit
- Handling user input
- Displaying parking status and records

Core Java concepts such as classes, objects, loops, conditional statements, and methods were used during development.

---

## 4.4Testing and Validation

Testing ensures that the program works correctly and produces the expected results.

Different test cases were used to check the functionality of the system.

Example Test Cases

| Test Case | Expected Result |
|---|---|
| Register vehicle | Vehicle stored correctly |
| Allocate slot | Slot assigned successfully |
| Vehicle entry | Entry recorded correctly |
| Vehicle exit | Exit recorded and slot freed |
| Invalid input | Error handled properly |

This phase ensures that the program runs without errors and produces accurate results.

---

## 4.5.Result and Documentation

After successful testing, the results of the system are documented. The system generates parking details such as allocated slot, entry time, exit time, and slot status.

The results are demonstrated through program execution and screenshots showing system input and output.

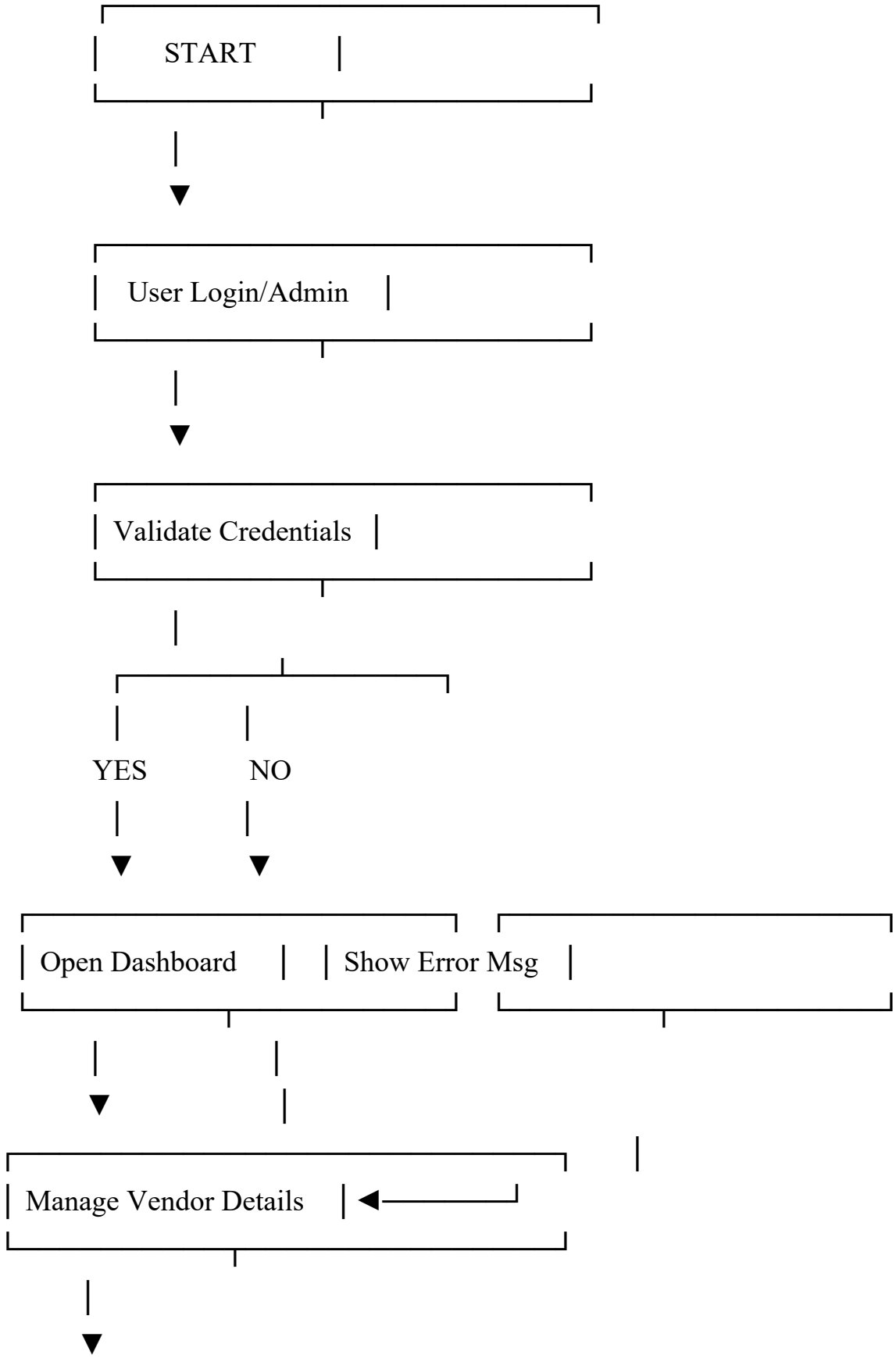
Proper documentation is prepared to explain the design, implementation, and working of the system.

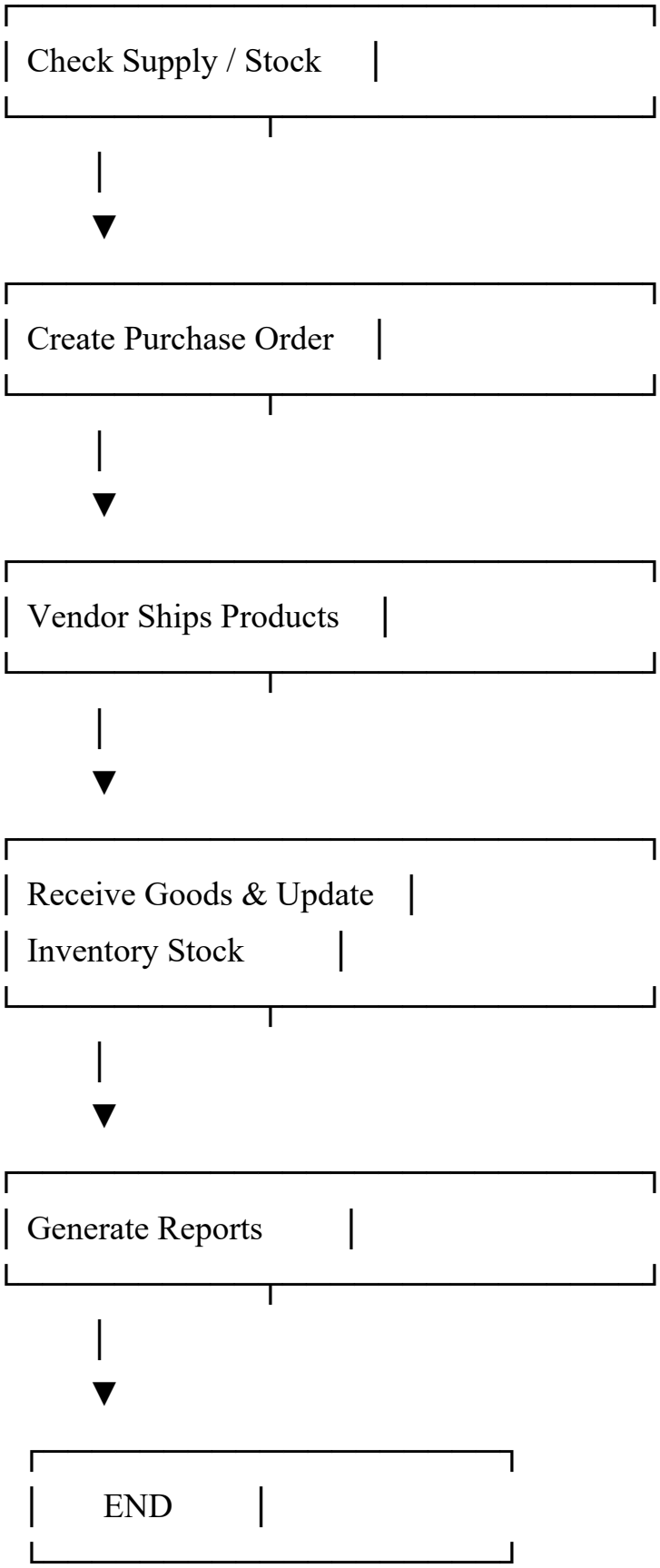
## 4.6.Algorithm

Algorithm for the Parking Slot Allocation and Vehicle Entry Tracking System:

1. Start the program
2. Display menu options
3. Register vehicle details
4. Check for available parking slots
5. Allocate slot to the vehicle
6. Record vehicle entry time
7. Update slot status as occupied
8. On vehicle exit, record exit time
9. Free the parking slot
10. Display parking details/status
11. End the program

4.7.Flowchart :




Check Supply / Stock
Create Purchase Order
Vendor Ships Products
Receive Goods & Update Inventory Stock
Generate Reports
END

# 5.SOURCE CODE

```
import java.util.Scanner;
import
java.util.ArrayList;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class ParkingSystem {

    static int totalSlots;
    static String[] slotVehicleNumber;
    static String[] slotOwnerName;
    static String[] slotVehicleType;
    static String[] slotEntryTime;
    static boolean[] slotOccupied;
    static ArrayList<String> history = new ArrayList<>();
    static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {

System.out.println("╔═══════════════════════════════════════════════════════
═══╗");
        System.out.println("║  PARKING SLOT ALLOCATION & TRACKING SYS  ║");
System.out.println("╚═══════════════════════════════════════════════════════
═══╝");

        System.out.print("\nEnter total number of parking slots: ");
```

```
totalSlots =
scanner.nextInt();
scanner.nextLine();

slotVehicleNumber = new
String[totalSlots]; slotOwnerName
                    = new String[totalSlots];
slotVehicleType  = new String[totalSlots];
slotEntryTime    = new String[totalSlots];
slotOccupied     = new boolean[totalSlots];

System.out.println("\n  Parking lot ready with " + totalSlots + " slots!\n");

while (true) {
   printMenu();

   System.out.print("Your choice: ");
   int choice = scanner.nextInt();
   scanner.nextLine();

   if     (choice == 1) parkVehicle();
   else if (choice == 2)
   removeVehicle(); else if (choice ==
   3) viewStatus(); else if (choice == 4)
   searchVehicle(); else if (choice == 5)
   viewHistory(); else if (choice == 6) {
      System.out.println("\n  Thank you! Goodbye.");
      break;
   }
   else {
      System.out.println("  Invalid choice! Enter a number from 1 to 6.");
```

```
        }
    }
}

static void printMenu() {
    System.out.println("\n========== MAIN MENU ==========");
    System.out.println("  1. Park a Vehicle");
    System.out.println("  2. Remove a Vehicle");
    System.out.println("  3. View Parking Status");
    System.out.println("  4. Search for a Vehicle");
    System.out.println("  5. View Entry / Exit History");
    System.out.println("  6. Exit");
    System.out.println("================================");
}

static void parkVehicle()
    { System.out.println("\n--- Park a Vehicle
    ---");

    System.out.print("  Vehicle Number       : ");
    String vehicleNumber = scanner.nextLine().toUpperCase().trim();

    System.out.print("  Owner Name           : ");
    String ownerName = scanner.nextLine().trim();

    System.out.print("  Type (Car/Bike/Truck) : ");
    String vehicleType = scanner.nextLine().trim();

    for (int i = 0; i < totalSlots; i++) {
        if (slotOccupied[i] && slotVehicleNumber[i].equalsIgnoreCase(vehicleNumber)) {
```

```
            System.out.println("  This vehicle is already parked at Slot " + (i + 1) + "!");
            return;
        }
    }

    for (int i = 0; i < totalSlots; i++)
        { if (!slotOccupied[i]) {
            slotOccupied[i]      = true;
            slotVehicleNumber[i] = vehicleNumber;
            slotOwnerName[i]     = ownerName;
            slotVehicleType[i]   = vehicleType;
            slotEntryTime[i]     = getCurrentTime();

            String log = "ENTRY | Slot " + (i + 1) +
                         " | [" + vehicleType + "] " + vehicleNumber +
                         " | Owner: " + ownerName +
                         " | Time: " + slotEntryTime[i];
            history.add(log);

            System.out.println("  SUCCESS! Vehicle parked at Slot " + (i + 1));
            return;
        }
    }

    System.out.println("  FULL! No slots available right now.");
}

static void removeVehicle()
    { System.out.println("\n--- Remove a Vehicle
    ---");
```

```
    System.out.print("  Enter Vehicle Number to remove: ");
    String vehicleNumber = scanner.nextLine().toUpperCase().trim();

    for (int i = 0; i < totalSlots; i++) {
        if (slotOccupied[i] && slotVehicleNumber[i].equalsIgnoreCase(vehicleNumber)) {

            String log = "EXIT  | Slot " + (i + 1) +
                         " | [" + slotVehicleType[i] + "] " + vehicleNumber
                         + " | Owner: " + slotOwnerName[i] +
                         " | Exit Time: " + getCurrentTime();
            history.add(log);

            slotOccupied[i]      = false;
            slotVehicleNumber[i] = null;
            slotOwnerName[i]     = null;
            slotVehicleType[i]   = null;
            slotEntryTime[i]     = null;

            System.out.println("  SUCCESS! Vehicle " + vehicleNumber +
                               " removed from Slot " + (i + 1));
            return;
        }
    }

    System.out.println("  Vehicle " + vehicleNumber + " not found in the parking lot.");
}

static void viewStatus()
    { int occupied  = 0;
```

```
    int available = 0;

    for (int i = 0; i < totalSlots; i++)
        { if (slotOccupied[i]) occupied+
        +; else available++;
    }

    System.out.println("\n========== PARKING LOT STATUS ==========");
    System.out.println("  Total: " + totalSlots
                    + "  |  Occupied: " + occupied
                    +  "  | Available: " +
                    available);
    System.out.println("----------------------------------------");

    for (int i = 0; i < totalSlots; i++)
        { if (slotOccupied[i]) {
            System.out.println("  Slot " + (i + 1) + " [OCCUPIED]"
                + "\n          Vehicle : " + slotVehicleNumber[i]
                + "\n          Owner   : " + slotOwnerName[i]
                + "\n          Type    : " + slotVehicleType[i]
                + "\n          Entry   : " + slotEntryTime[i]);
        } else {
            System.out.println("  Slot " + (i + 1) + " [AVAILABLE]");
        }
    }

    System.out.println("========================================");
}

static void searchVehicle() {
```

```
    System.out.println("\n--- Search Vehicle ---");
    System.out.print("  Enter Vehicle Number: ");
    String vehicleNumber = scanner.nextLine().toUpperCase().trim();

    for (int i = 0; i < totalSlots; i++) {
        if (slotOccupied[i] && slotVehicleNumber[i].equalsIgnoreCase(vehicleNumber))
            { System.out.println("  FOUND at Slot " + (i + 1) + "!"
              + "\n    Vehicle : " + slotVehicleNumber[i]
              + "\n    Owner   : " + slotOwnerName[i]
              + "\n    Type    : " + slotVehicleType[i]
              + "\n    Entry   : " + slotEntryTime[i]);
            return;
        }
    }

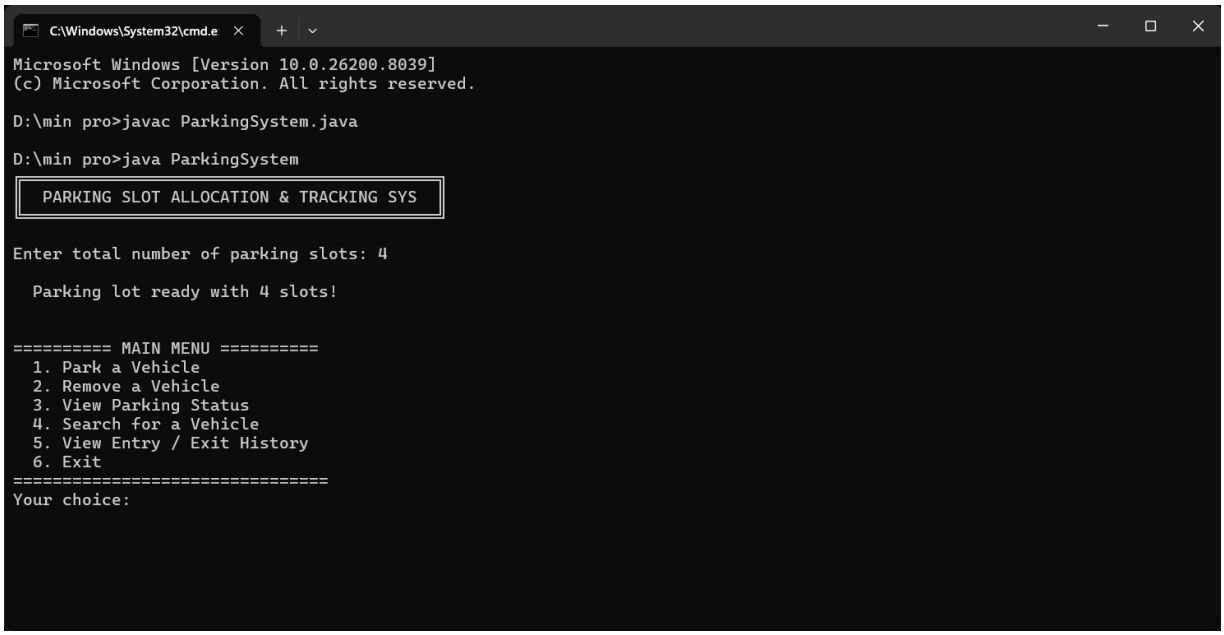
    System.out.println("  Vehicle " + vehicleNumber + " is not currently parked here.");
}

static void viewHistory() {
    System.out.println("\n========== VEHICLE HISTORY LOG ==========");
    if (history.isEmpty())
        { System.out.println("  No records
        yet.");
    } else {
        for (int i = 0; i < history.size(); i++)
            { System.out.println("  " + (i + 1) + ". " +
            history.get(i));
        }
    }
    System.out.println("=========================================");
}
```

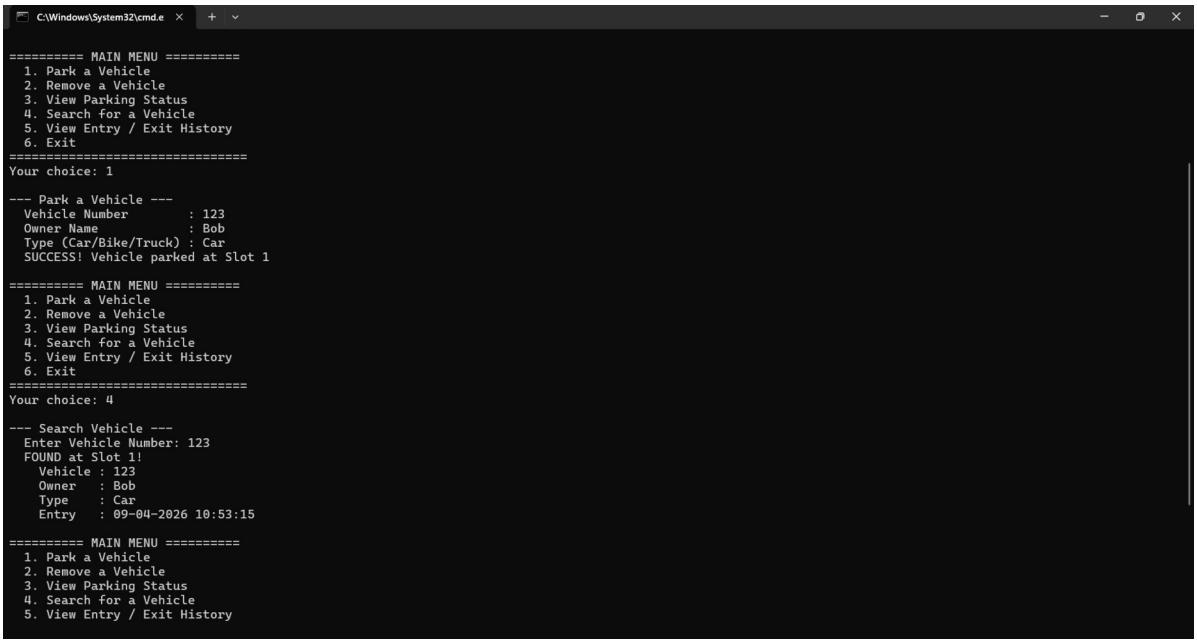
```
    static String getCurrentTime() {
        DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
        return LocalDateTime.now().format(fmt);
    }
}
```

# 6 . INPUT AND OUTPUT SCREENS

## INPUT_SCREEN:




OUTPUT SCREEN:



# 7. DISCUSSION & CONCLUSION

## 7.1.Discussion

The Parking Slot Allocation and Vehicle Entry Tracking System was developed to automate the process of managing parking slots and tracking vehicle entry and exit efficiently. The system allows users to register vehicle details, allocate parking slots, and maintain records of vehicle movement in an organized manner.

During the development process, Core Java programming concepts such as loops, conditional statements, methods, and object-oriented principles were applied. The system successfully performs parking slot allocation, tracks entry and exit details, and updates slot availability accurately.

The system was tested with different inputs to ensure that it performs all required operations correctly. The results show that the system minimizes manual effort, reduces errors, and improves the overall efficiency of parking management.

---

## 7.2Conclusion

The Parking Slot Allocation and Vehicle Entry Tracking System was successfully developed to provide an automated solution for managing parking operations. The system enables efficient allocation of parking slots, accurate tracking of vehicle entry and exit, and proper maintenance of parking records.

By using Core Java concepts such as classes, objects, loops, and conditional statements, the application demonstrates a practical implementation of programming knowledge in solving real-world problems.

The system was tested under various conditions and produced accurate results. It helps reduce manual workload, improves accuracy, and ensures better utilization of parking space.

Overall, this project highlights how Java programming can be used to develop simple, effective, and reliable applications for parking management systems.

# 8.REFERENCES

The following resources were used during the development of this project:


### Books

- Herbert Schildt – *Java: The Complete Reference*

### Websites

- https://www.oracle.com/java
- https://www.geeksforgeeks.org
- https://www.w3schools.com
- https://www.javatpoint.com